

Information Retrieval Midterm Exam

Questions and Answers

Azarbaijan Shahid Madani University - Second Semester of
Academic Year 2025-2026

Question 1

Considering the following lists:

$$w1 : \langle 1, 2, 3 \rangle$$
$$w2 : \langle 4, 5 \rangle$$
$$w3 : \langle 4, 5 \rangle$$

What is the proper and optimal order to process the following query?

$w1 \text{ AND } w2 \text{ AND } w3$

Answer:

To optimize the processing of queries combined with the **AND** operator, we must sort the lists in ascending order based on their lengths (document frequency). The reason for this is that the intersection of two small lists produces a smaller temporary list, drastically reducing the cost of subsequent comparisons.

According to the problem data:

- The length of list $w1$ is 3.
- The length of list $w2$ is 2.
- The length of list $w3$ is 2.

Therefore, the optimal processing order is to first intersect **w2** and **w3**, and then intersect the result with **w1**:

$(w2 \text{ AND } w3) \text{ AND } w1$ or $(w3 \text{ AND } w2) \text{ AND } w1$

Question 2

How many comparisons are required to merge the following posting lists? Write down the list of comparisons. (Note that in the first list, skip pointers of length 8 are used, and DOC IDs that have skip pointers are specified in **bold**.)

(Note that D is equal to the last digit of your student ID number.)

List1: $\langle \mathbf{3}, 5, 9, 15, 24, 39, 60, (\mathbf{70}+D), 81, 84, 89, 92, 96, 97, 100, \mathbf{115} \rangle$ skip=8

List2: $\langle \mathbf{2}, 12, 75, 76, 77, 78, 89, 95, \mathbf{97}, 99, 100, 101 \rangle$

Answer:

The last digit of student ID 4011833230 is zero ($D = 0$). Therefore, the middle bold element in the first list is the number **70**.

According to the question's assumption, the lists are as follows, where the bold elements indicate Skip pointers:

- L1: $\langle \mathbf{3}, 5, 9, 15, 24, 39, 60, \mathbf{70}, 81, 84, 89, 92, 96, 97, 100, \mathbf{115} \rangle$
- L2: $\langle \mathbf{2}, 12, 75, 76, 77, 78, 89, 95, \mathbf{97}, 99, 100, 101 \rangle$

In the merge algorithm, when an element of one list is smaller than the other, we check whether the current node has a skip pointer. If it does, we **compare** the skip target with the element of the other list to see if the skip is allowed (this check itself counts as a comparison).

The sequence of 27 comparisons is as follows (using p_1 for the first list pointer and p_2 for the second list pointer):

1. Comparison (3 and 2) $\rightarrow 3 > 2$. (Element 2 has a skip to 97)
2. **Skip check** (3 and 97) $\rightarrow 97 \not\leq 3$. Unsuccessful skip, p_2 moves to 12.
3. Comparison (3 and 12) $\rightarrow 3 < 12$. (Element 3 has a skip to 70)
4. **Skip check** (70 and 12) $\rightarrow 70 \not\leq 12$. Unsuccessful skip, p_1 moves to 5.
5. Comparison (5 and 12) $\rightarrow 5 < 12 \rightarrow p_1$ moves to 9.
6. Comparison (9 and 12) $\rightarrow 9 < 12 \rightarrow p_1$ moves to 15.
7. Comparison (15 and 12) $\rightarrow 15 > 12 \rightarrow p_2$ moves to 75.
8. Comparison (15 and 75) $\rightarrow 15 < 75 \rightarrow p_1$ moves to 24.
9. Comparison (24 and 75) $\rightarrow 24 < 75 \rightarrow p_1$ moves to 39.
10. Comparison (39 and 75) $\rightarrow 39 < 75 \rightarrow p_1$ moves to 60.
11. Comparison (60 and 75) $\rightarrow 60 < 75 \rightarrow p_1$ moves to 70.
12. Comparison (70 and 75) $\rightarrow 70 < 75$. (Element 70 has a skip to 115)
13. **Skip check** (115 and 75) $\rightarrow 115 \not\leq 75$. Unsuccessful skip, p_1 moves to 81.
14. Comparison (81 and 75) $\rightarrow 81 > 75 \rightarrow p_2$ moves to 76.

15. Comparison (81 and 76) $\rightarrow 81 > 76 \rightarrow p_2$ moves to 77.
16. Comparison (81 and 77) $\rightarrow 81 > 77 \rightarrow p_2$ moves to 78.
17. Comparison (81 and 78) $\rightarrow 81 > 78 \rightarrow p_2$ moves to 89.
18. Comparison (81 and 89) $\rightarrow 81 < 89 \rightarrow p_1$ moves to 84.
19. Comparison (84 and 89) $\rightarrow 84 < 89 \rightarrow p_1$ moves to 89.
20. Comparison (89 and 89) $\rightarrow$ **Equal (found)**. p_1 moves to 92 and p_2 moves to 95.
21. Comparison (92 and 95) $\rightarrow 92 < 95 \rightarrow p_1$ moves to 96.
22. Comparison (96 and 95) $\rightarrow 96 > 95 \rightarrow p_2$ moves to 97.
23. Comparison (96 and 97) $\rightarrow 96 < 97 \rightarrow p_1$ moves to 97.
24. Comparison (97 and 97) $\rightarrow$ **Equal (found)**. p_1 moves to 100 and p_2 moves to 99.
(Since they were equal, the skip check on 97 is not performed).
25. Comparison (100 and 99) $\rightarrow 100 > 99 \rightarrow p_2$ moves to 100.
26. Comparison (100 and 100) $\rightarrow$ **Equal (found)**. p_1 moves to 115 and p_2 moves to 101.
27. Comparison (115 and 101) $\rightarrow 115 > 101 \rightarrow p_2$ reaches the end of the list.

Total comparisons: 27 comparisons.

Found common elements: 89, 97, 100.

Question 3

In an IR system, what impact can each of the following have on Precision and Recall for an arbitrary query? Write your answer with an example and reason.

1. Performing stemming and lemmatization

Answer:

Generally, both methods attempt to reduce words to their root or base form.

- **Impact on Recall: Increases.** Since different forms of a word (such as plurals, various verb forms) are mapped to a single root, more documents will be retrieved. For example, if a user searches for the query *operate*, due to stemming, documents containing *operating*, *operation*, *operative* will also be retrieved, ensuring no relevant document is missed.
- **Impact on Precision: Decreases.** Due to merging words with the same root but different meanings, irrelevant documents may be displayed to the user. For instance, the word *operating* (operating system in computers) and *operative* (operational / medical) both map to the root *oper* in Stemming. This causes irrelevant articles to be returned when searching for computer papers, dropping the precision.

Note: Compared to stemming, lemmatization has higher *Precision* and causes less damage because it uses a dictionary to find the actual semantic form of the word.

Question 4

Calculate the edit distance between your name word (e.g., *Ali*) and the word *name* using the Levenshtein method. (Note that in this method only three operations—insertion, deletion, and substitution—are allowed).

Answer:

Assumed name: *ali* (in lowercase for simplicity).

Target word: *name*.

In the Levenshtein method, the cost of deletion, insertion, and substitution operations is considered to be 1. The dynamic programming matrix for these two words is constructed as follows:

		a	l	i
n	0	1	2	3
a	1	1	2	3
m	2	1	2	3
e	3	2	2	3

The value of the bottom-right cell of the matrix is **3**. Therefore, the edit distance is 3.

Conversion steps (*ali* → *name*):

1. **Insert** letter *n* at the beginning of the word → *nali* (cost: 1)
2. Letter *a* matches letter *a* → no change (cost: 0)
3. **Substitute** letter *l* with *m* → *nami* (cost: 1)
4. **Substitute** letter *i* with *e* → *name* (cost: 1)

Total cost: 3 operations.

Question 5

Determine whether the following statements are true or false.

- (a) Stemming causes an increase in the dictionary size in an inverted index.
- (b) Using skip pointers speeds up the processing of an *X OR Y* query.

Answer:

(a) **False (Incorrect).** Stemming causes different forms of a word to map to a single root (such as *play*). This reduces the number of unique words (vocabulary) and consequently results in a **decrease** in dictionary size.

(b) **False (Incorrect).** Skip pointers are solely used to speed up intersection computations and *AND* queries. In processing an *OR* query, the union of two lists must be computed, meaning all elements of both lists must be traversed and merged; therefore, skipping elements is not possible and does not speed it up.

Question 6

Suppose an information retrieval system is given with the following dictionary:

Term ID	Term
1	retried
2	removed
3	relieved
4	retrieved
5	redirected
6	remained

a) What steps must be taken to answer the query **iev** using 3-grams? And what words from the list are the answer to this query? Write your answer accurately.

Answer:

Steps to process the query **iev:**

1. **Query Analysis:** Since there are *** characters at the beginning and end of the query, we do not need to add start and end markers (*\$*). Therefore, the only 3-gram present in this query is the string *iev*.
2. **Extracting 3-grams and searching the index:** In the 3-gram index, we look for the posting list corresponding to the key *iev*. By checking the dictionary words, we see which words contain *iev*:
 - 1: retried → does not have it
 - 2: removed → does not have it
 - **3: relieved** → contains *iev* relieved
 - **4: retrieved** → contains *iev* retrieved
 - 5: redirected → does not have it
 - 6: remained → does not have it

Therefore, the initial candidate list of documents (words) will be IDs **3 and 4**.

3. **Post-filtering:** We fetch the words retrieved in the previous step and check them as strings against the query regular expression (here, matching the substring *iev*). Since both words *relieved* and *retrieved* explicitly contain *iev*, they pass this filter.

Final result: The words *relieved* (ID 3) and *retrieved* (ID 4) are the answers to this query.

Question 7

In the logarithmic merge method, if the size of the in-memory auxiliary index is $Z_0 = 8$ and we have the indices I_3, I_2, I_1, I_0 . Assuming the number of tokens is 65, perform step-by-step simulation of the logarithmic merge algorithm.

Answer:

The temporary memory capacity is $Z_0 = 8$ tokens. The total number of tokens is $N = 65$.

The capacities of the different index levels in logarithmic merge are as follows:

- Capacity of $I_0 = 1 \times Z_0 = 8$
- Capacity of $I_1 = 2 \times Z_0 = 16$
- Capacity of $I_2 = 4 \times Z_0 = 32$
- Capacity of $I_3 = 8 \times Z_0 = 64$

We have $65 \div 8 = 8$ full blocks (each 8 tokens) and 1 extra token remaining. The binary representation of the number 8 is *1000*, which shows that ultimately only level I_3 will be full and the rest of the disk levels will remain empty.

Step-by-step simulation:

1. **Tokens 1 to 8:** Memory Z_0 fills up. Since I_0 is empty, Z_0 is written to disk and I_0 is created (size 8). Z_0 becomes empty.
2. **Tokens 9 to 16:** Memory Z_0 fills up. I_0 is already full. Thus, Z_0 and I_0 are merged to form I_1 (size 16). I_0 and Z_0 become empty.
3. **Tokens 17 to 24:** Memory Z_0 fills up. I_0 is empty, so it is written to I_0 (size 8).
4. **Tokens 25 to 32:** Memory Z_0 fills up. I_0 is full. Merging of Z_0 and I_0 occurs, and the result should go to I_1 . But I_1 is also full. Thus, Z_0, I_0, I_1 are all merged together to form I_2 (size 32). Z_0, I_0, I_1 all become empty.
5. **Tokens 33 to 40:** Z_0 fills up. I_0 is created (size 8).
6. **Tokens 41 to 48:** Z_0 fills up. It merges with I_0 to form I_1 (size 16).
7. **Tokens 49 to 56:** Z_0 fills up. I_0 is created (size 8).

8. **Tokens 57 to 64:** Z_0 fills up. I_0, I_1, I_2 are all full. All of these (Z_0, I_0, I_1, I_2) undergo a cascading merge to form I_3 (size 64). All lower levels become empty.
9. **Token 65:** Enters the temporary memory Z_0 .

Final system status: I_3 has 64 tokens (full), I_2 is empty, I_1 is empty, I_0 is empty, and the memory Z_0 has 1 token.